

Edit History
** Changes in Version 20220531**
Deprecated mAPI.installApkSilently("/sdcard/API.apk"); function
Add `mAPI.installApkSilently("/sdcard/API.apk", "package name", "class name");` to automatically launch the application's API based on the package name and class name.
20220621 Category Interface Description
** Added interfaces for setting serial number, setting/getting device model, setting/getting Ethernet MAC address, and setting/getting IMEI number in 20220625 **
** Added Ethernet switch and status acquisition in 20220630**
** Added APP Guardian Interface on July 1, 2022**
** Added settings for boot image and boot animation on July 13, 2022. , Set the status of the drop-down status bar**
** Get screen on/off status at 20220830**
** Modified the timed power on/off interface on September 22, 2022, adding several modes.**
** Added APP foreground protection interface on 20221107**
** Added separate control for screen display on/off in 20230413**
** Modified the description of the Wiegand asynchronous read interface function on 20240321**
** Added the setInstallPackagePolicy interface on May 21, 2024, to set the APK installation policy (whitelist).**
** Added language interface settings in 20250310**
** Added watchdog timeout setting on 20251224**

API Interface Description

- The API interfaces described in this document correspond to our company's products. The supported hardware interfaces need to be determined by referring to our company's hardware specifications.
- To test the effectiveness of the relevant API calls, go to 'Settings' - 'User Settings', or 'Settings' - 'User Settings' - 'YNHCommonAPI' and select the corresponding function item.
- If the documentation is incomplete or contains missing parameters, please refer to the YNHCommonAPI source code.

1: Obtain an API instance

Example code:

```
// Get the unique instance of
the process YNHAPI mAPI = YNHAPI.getInstance();
```

Note: All subsequent example code uses this instance object.

2: System Parameters

2.1: Obtain the motherboard model

Example code:

```
// Get the motherboard model , returning the specific board model such as ynh960, ynh530,  
etc. String model = mAPI.getBoardModel();
```

2.2: Serial Number

2.2.1: Obtain the serial number

Example code:

```
// Get serial number.  
String sn = mAPI.getSerialNo(); // The return result is consistent with Build.SERIAL
```

2.2.2: Setting the serial number

Example code:

```
// Set the serial number. After setting, you need to call the restart function for  
the changes to take effect. String SerialNo =  
"12345678"; mAPI.setSerialNo(SerialNo);  
mAPI.reboot();
```

2.3: Ethernet MAC Address

2.3.1: Obtain the Ethernet MAC address

Example code:

```
// Get Ethernet MAC address.  
String mac = mAPI.getEthernetMAC(); // The returned result is consistent with the Ethernet value returned by the network service module.
```

2.3.2: Setting the Ethernet MAC Address

Example code:

```
// Set the Ethernet MAC address. A reboot function must be called after setting for the  
changes to take effect. String mac = "00:E0:EC:37:2A:42";  
mAPI.setEthernetMAC(mac); mAPI.reboot();
```

2.4: 4G Module IMEI

2.4.1: Obtain the 4G module IMEI

Example code:

```
// Get the IMEI of the 4G module.  
String imei = mAPI.getIMEI(); // The returned result is consistent with the Ethernet value returned by the network service module.
```

2.4.2: Setting the 4G module IMEI

Example code:

```
// Set the 4G module IMEI. After setting, a restart function needs to be called for the  
changes to take effect. String imei = "864793030776509";  
mAPI.setIMEI(imei); mAPI.reboot();
```

2.5: Equipment Model

2.5.1: Obtain the device model

Example code:

```
// Get device model.  
String model = mAPI.getProductModel(); // The return result is consistent with Build.MODEL
```

2.5.2: Set the device model

Example code:

```
// Set the device model. After setting, a restart function must be called for the  
changes to take effect. String model =  
"123456"; mAPI.setProductModel(model); mAPI.reboot();
```

2.6: Device Storage Information

Related data class description:

```
//Device storage information (path,
space size, etc.) public class
StorageInfo{ public final static int TYPE_MEMORY = 1;
public final static int TYPE_LOCAL_STORAGE = 2; public final static
int TYPE_TFCARD = 3; public final static int
TYPE_USB_STORAGE = 4; private int mType = 0; private long
mTotalSize = 0; //Unit: KB
private long mFreeSize = 0; //Unit: KB private String
mPath = "";

}
```

Example code:

```
// Get device storage information, including internal storage (EMMC), TF/SD card, USB
storage (USB flash drive, external hard drive, etc.) List<StorageInfo>
list = mAPI.getStorageInfos();
for(StorageInfo info : list){ }
```

2.7: Setting the boot image

Example code:

```
mAPI.setBootLogo("/sdcard/boot.bmp"); // The boot image must be an 8-bit BMP bitmap.
```

2.8: Setting the boot animation

Example code:

```
mAPI.setBootAnimation("/sdcard/bootanimation.zip"); // The boot animation must be a standard Android animation; the name of the zip archive
can be changed arbitrarily.
```

2.9: Configure APK installation policy (whitelist)

```
boolean setInstallPackagePolicy(YNHAPI.InstallPackagePolicy policy);
```

parameter:

YNHAPI.InstallPackagePolicy policy: The APK installation policy (whitelist) that needs to be set. See the example code below for parameters.

Example code:

```
YNHAPI.InstallPackagePolicy policy = new YNHAPI.InstallPackagePolicy();

List<String> allowInstallPackageList = new ArrayList<>(); int allowInstallType
= 0;
String allowInstallPassword = null;

allowInstallPackageList.add("com.test.apk"); allowInstallType
= 0; allowInstallPassword =
"1234abcd";

/** *
    A whitelist of allowed APKs for installation. The `allowInstallPackageList` is empty if it is, meaning all APKs are prohibited from installation. */
A password is required to install.
/
policy.setAllowInstallPackageList(allowInstallPackageList); /** * A whitelist of
allowed APKs for installation. *
`mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_NORMAL` indicates that installation is disabled. */
set up
    * `mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_NOT_ALLOW` indicates that only APKs listed in `mAllowInstallPackageList`
are allowed to be installed; otherwise, an installation rejection message will be displayed. *
    `mAllowInstallType = YNHAPI.InstallPackagePolicy.ALLOW_INSTALL_TYPE_PASSWORD` indicates that only APKs listed in `mAllowInstallPackageList`
are allowed to be installed; otherwise, a password prompt will appear, requiring the user to enter a password
(`mAllowInstallPassword`). Installation can continue only after successful password verification.
/
policy.setAllowInstallType(allowInstallType); /** * Allows the
password to be used when installing
APKs */
policy.setAllowInstallPassword(allowInstallPassword);

mAPI.setInstallPackagePolicy(YNHAPI.InstallPackagePolicy policy);
```

2.10: Setting the Language

Example code:

```
String language="zh"; String
country="CN";
mAPI.updateLanguage(language, country);
```

Commonly used Chinese characters (China) are: values-zh-rCN

Chinese (Taiwan): values-zh-r

Chinese (Hong Kong): values-zh-rHK

English (United States): values-en-rUS

English (United Kingdom): values-en-rGB

English (Australia): values-en-rAU
English (Canada): values-en-rCA
English (Ireland): values-en-rIE English
(India): values-en-rIN English (New
Zealand): values-en-rNZ English
(Singapore): values-en-rSG English
(South Africa): values-en-rZA

Arabic (Egypt): values-ar-rEG Arabic
(Israel): values-ar-rIL **Bulgarian: values-
bg-rBG** Catalan: values-ca-rES
Czech: values-cs-rCZ Danish :
values-da-rDK German
(Austria): values-de-rAT
German (Switzerland): values-de-rCH
German (Germany): values-de-rDE
German (Liechtenstein): values-de-
rLI Greek: values-el-rGR Spanish (Spain):
values-es-rES Spanish
(USA): values-es-rUS Finnish (Finland):
values-fi-rFI French (Belgium): values-fr-
rBE French (Canada): values-fr-rCA
French (Switzerland): values-fr-rCH
French (France): values-fr-rFR

Hebrew: values-iw-rIL Hindi:
values-hi-rIN Croatian:
values-hr-rHR Hungarian: values-
hu-rHU Indonesian: values-in-
rID Italian (Swiss): values-it-rCH

Italian: values-it-rIT Japanese: values-ja-
rJP Korean: values-ko-
rKR
Lithuanian: valueslt-rLT;
Latvian: values-lv-rLV; Norwegian
Bokmål: values-nb-rNO; Dutch
(Belgium): values-nl-BE; Dutch
(Netherlands): values-nl-rNL; Polish:
values-pl-rPL ; Portuguese
(Brazil): values-pt-rBR; Portuguese
(Portugal): values-pt-rPT ; Romanian:
values-ro-rRO; Russian: values-
ru-rRU; Slovak: values-sk-
rSK ; Slovenian: values-sl-rSI;
Serbian: values-sr-rRS

Swedish: values-sv-rSE
Thai: values-th-rTH
Tagalog: values-tl-rPH
Turkish: values--r-rTR
Ukrainian: values-uk-rUA

Vietnamese: values-vi-rVN

3: OTA upgrade

```
void otaUpdate(String otaFilePath)
```

parameter:

- otaFilePath: Path to the ota package.

Note: The upgrade package must be located in the sdcard directory and named update.zip.

Example code:

```
mAPI.otaUpdate("/sdcard/update.zip");
```

4: APK

4.1: Applications start automatically on system boot

```
void setBootLaunchApk(String packageName, boolean launch)
```

parameter:

- packageName: The name of the application package that needs to start automatically on boot.
- `launch: true` means the device will start automatically on boot. `false` means the device will not start automatically on boot.

Example code:

```
mAPI.setBootLaunchApk("com.innohi.helloworld", true);
```

#

4.2: Silently install APK, automatically start after installation.

```
void installApkSilently(String apkPath,String packageName,String className)
```

parameter:

- apkPath: The path to the APK.
- packageName: The full package name of the APK to be launched. For example, APK package name com.example
- className: The fully qualified class name of the APK to be launched. For example, APK class name com.example.MainActivity

Example code:

// Silently install

```
APK mAPI.installApkSilently("/sdcard/API.apk", "com.example", "com.example.MainActivity");
```

If the last two parameters are null, it means that the silently installed application does not need to be launched automatically.

```
mAPI.installApkSilently("/sdcard/API.apk", null, null);
```

4.3: Uninstall APK

```
void uninstallApkSilently(String packageName)
```

parameter:

- packageName: The package name of the APK.

Example code:

```
mAPI.uninstallApkSilently("com.innohi.api");
```

4.4: Protect the APK

A: Non-foreground guardian

```
void setAppKeepLive(String packageName, int keepAliveTimeSec)
```

Reboot does not retain, meaning the APK needs to be called again after reboot.

parameter:

- packageName: The package name of the APK.
- keepAliveTimeSec: APK detection timeout, in seconds. If the APK is not detected as alive after this time, the APK will be manually restarted.

Example code:

```
mAPI.setAppKeepLive("com.innohi.api", 10); // Cancel  
non-foreground  
daemon mAPI.setAppKeepLive("", 1);
```

B: Front Desk Guardian

```
void setForegroundAppKeepLive(String packageName, int keepAliveTimeSec)
```

Reboot does not retain, meaning the APK needs to be called again after reboot.

parameter:

- packageName: The package name of the APK.

- `keepAliveTimeSec`: APK detection timeout, in seconds. If the APK is not detected as alive after this time, the APK will be manually restarted.

Example code:

```
mAPI.setForegroundAppKeepLive("com.innohi.api", 10); // Cancel foreground app keeper mAPI.setForegroundAppKeepLive("", 1);
```

5: Display related information

5.1: Obtain the screen rotation angle

`RotationDegree getScreenRotation(ScreenType screenType)`

parameter:

- `screenType`: Indicates which screen's rotation angle to retrieve. For example, `ScreenType.MAIN` represents the main screen, and `ScreenType.AUX` represents the secondary screen.

Return value:

- `RotationDegree`: Returns the rotation angle of the specified screen. For example, `RotationDegree.DEGREE_90` represents 90 degrees.

Example code:

```
YNHAPI.RotationDegree screenRotation
    = mAPI.getScreenRotation(YNHAPI.ScreenType.MAIN);
If (screenRotation == YNHAPI.RotationDegree.DEGREE_90) { Log.d(TAG,
    "Main screen 90 degrees");
}
```

5.2: Set screen rotation angle

`void setScreenRotation(ScreenType type, RotationDegree degree)`

parameter:

- `type`: Sets the rotation angle of which screen. For example, `ScreenType.MAIN` represents the home screen.
- `degree`: The angle to be set. For example, `RotationDegree.DEGREE_90` means 90 degrees.

Example code:

```
// Set the secondary screen rotation by 180 degrees.
mAPI.setScreenRotation(YNHAPI.ScreenType.AUX, YNHAPI.RotationDegree.DEGREE_180);
```

5.3: Obtaining Screen Density

`LcdDensity` `getLcdDensity()`

Return value:

- `LcdDensity`: Screen density. For example, `LcdDensity.DENSITY_160` indicates a screen density of 160 dpi.

Example code:

```
YNHAPI.LcdDensity density = mAPI.getLcdDensity(); if (density ==  
YNHAPI.LcdDensity.DENSITY_160) { Log.d(TAG, "Screen  
    density is 160dpi");  
}
```

5.4: Setting Screen Density

`void` `setLcdDensity(LcdDensity density)`

parameter:

- `density`: Indicates the screen density to be set. For example, `LcdDensity.DENSITY_160` means setting the screen density to 160 dpi.

Example code:

```
// Set the screen density  
to 240dpi mAPI.setLcdDensity(YNHAPI.LcdDensity.DENSITY_240);
```

5.5: Obtain screen on/off status

`boolean` `isScreenOn()`

parameter:

- `none`
- Cannot be used to determine screen on/off status after sleep/wakeup.
- This can be used by an app to call `setScreenOnOff` to customize the screen's on/off state.

Example code:

```
// True for screen on, false for  
screen off mAPI.isScreenOn();
```

5.6: Setting the screen on/off switch

```
void setScreenOnOff(boolean on)
```

parameter:

- boolean on Whether to display on the screen
- For non-standard sleep/wake-up scenarios, the app can control the screen's on/off state independently.
- Determine the screen on/off state using isScreenOn

Example code:

```
// Sets `true` for screen on, `false`  
for screen off. `mAPI.setScreenOnOff(false)`
```

6: Watchdog

6.1: Turning the watchdog on/off

```
void enableWatchdog(boolean enable)
```

parameter:

- enable: true indicates that the watchdog timer is enabled, and false indicates that the watchdog timer is disabled.

Example code:

```
// Enable  
watchdog ( mAPI.enabledWatchdog(true); //  
Disable  
watchdog (mAPI.enableWatchdog(false);)
```

6.2: Feeding the dog

```
void feedWatchdog()
```

Example code:

```
// Feed the  
watchdog // After activating the watchdog function, it will automatically reset within 15 seconds if the watchdog is not fed. It is recommended to call the `feedWatchdog` function approximately  
every 10 seconds to feed the watchdog. `mAPI.feedWatchdog();`
```

6.3: Setting the watchdog timeout

```
void setWatchdogTimeout(int second)
```

Example code:

```
// Sets a timeout after 15 seconds. The timeout may vary by 1-2 seconds
depending on the platform. mAPI.setWatchdogTimeout(15);
```

#

7: Shutting down and restarting

Example code:

```
//
Shutdown

mAPI.shutdown(); // Reboot mAPI.reboot();
```

8: System hibernation and wake-up

Example code:

```
// Determine system hibernation/wake status using PowerManager
PowerManager pm = (PowerManager) mSystemContext.getSystemService(Context.POWER_SERVICE); boolean
isInteractive = pm.isInteractive(); //Android 11 and later Android interface // Sleep mAPI.sleep(); //
Wake
up mAPI.wakeup();
```

9: Navigation bar and drop-down status bar

9.1: Setting Navigation Bar Visibility

```
void setNavigationBarVisibility(NavigationBarVisibility visibility)
```

parameter:

- visibility: Navigation bar visibility. There are three possible values, as follows:
 - NavigationBarVisibility.INVISIBLE: The navigation bar is not visible, but it can be swiped out from the bottom of the screen.
 - NavigationBarVisibility.VISIBLE: Makes the navigation bar visible.
 - NavigationBarVisibility.ALWAYS_INVISIBLE: The navigation bar is invisible and cannot be swiped out from the bottom of the screen.

Example code:

```
mAPI.setNavigationBarVisibility(YNHAPI.NavigationBarVisibility.ALWAYS_INVISIBLE);
```

9.2: Obtaining Navigation Bar Visibility

NavigationBarVisibility getNavigationBarVisibility()

Return value:

- NavigationBarVisibility: The visibility of the navigation bar. There are three possible values, as follows:
 - NavigationBarVisibility.INVISIBLE: The navigation bar is not visible, but it can be swiped out from the bottom of the screen.
 - NavigationBarVisibility.VISIBLE: Makes the navigation bar visible.
 - NavigationBarVisibility.ALWAYS_INVISIBLE: The navigation bar is invisible and cannot be swiped out from the bottom of the screen.

Example code:

```
// Get navigation bar visibility.  
YNHAPI.NavigationBarVisibility visibility = mAPI.getNavigationBarVisibility(); // If the navigation bar is  
always invisible, then display it. if (visibility  
== YNHAPI.NavigationBarVisibility.ALWAYS_INVISIBLE) {  
    mAPI.setNavigationBarVisibility(YNHAPI.NavigationBarVisibility.VISIBLE);  
}
```

9.3: Setting the visibility of the drop-down status bar

void setExtendStatusBarVisibility(ExtendStatusBarVisibility visibility)

Example code:

```
// After setting, you need to restart the device for the settings  
to take effect after the next boot. mAPI.setExtendStatusBarVisibility(YNHAPI.ExtendStatusBarVisibility.VISIBLE); //The top status bar can  
be pulled down.
```

9.4: Get the visibility of the dropdown status bar

ExtendStatusBarVisibility getExtendStatusBarVisibility()

Return value:

- ExtendStatusBarVisibility: Status bar visibility. There are two possible values, as follows:
 - ExtendStatusBarVisibility.INVISIBLE: The status bar cannot be pulled down.
 - ExtendStatusBarVisibility.VISIBLE: Enables a dropdown menu for the status bar.

Example code:

```
// Get the visibility of the dropdown status bar.  
YNHAPI.ExtendStatusBarVisibility visibility = mAPI.getExtendStatusBarVisibility();
```

10: Root privileges

10.1: Detecting root access

```
boolean isRoot()
```

Return value:

- ``boolean : true`` indicates that root privileges can be obtained using ``su``. Otherwise, they cannot.

10.2: Enable/Disable Root Access

```
boolean enableRoot(boolean enable)
```

parameter:

- ``enable: true`` means root access is enabled. ``false`` means root access is disabled.

Return value:

- `boolean: true`, indicating success; `false`, indicating failure.

Example code:

```
// If root privileges are disabled, enable
root privileges if (!mAPI.isRoot()) {
    if (mAPI.enableRoot(true)) { Log.d(TAG,
        "Successful root");
    }
}
```

11: Network-related

11.1: Ethernet

11.1.1: Obtain IP Information

```
IpConfig getIpConfig()
```

Return value:

- `IpConfig`: An entity class representing IP information.

Example code:

```
// Get IP
information IpConfig ipConfig = mAPI.getIpConfig(); //
IP address
String ip = ipConfig.ip; // Subnet
mask
String mask = ipConfig.mask; //
Gateway
String gateway = ipConfig.gateway; // dns list

ArrayList<String> dnsList = ipConfig.dnsList;
```

11.1.2: Using a static Ethernet IP

```
void setStaticIp(IpConfig config)
```

parameter:

- config: This indicates that information about setting a static IP address needs to be configured.

Example code:

```
mAPI.setStaticIp(new IpConfig("192.168.1.2", "255.255.255.0", "192.168.8.1", Arrays.asList("192.168.8.1")));
```

11.1.3: Using Ethernet Dynamic IP

```
void setDhcpIp()
```

Example code:

```
// Set to use
dynamic IP mAPI.setDhcpIp();
```

11.1.4: Obtain Ethernet IP Mode

```
IpMode getIpMode()
```

Return value:

- IpMode: Indicates the IP mode. For example, IpMode.STATIC indicates static IP mode, and IpMode.DHCP indicates dynamic IP mode.

Example code:

```
`IpMode mode = mAPI.getIpMode(); if
(mode == IpMode.STATIC)
    { Log.d(TAG, "Static IP
mode"); } else if (mode == IpMode.DHCP)
    { Log.d(TAG, "Dynamic IP mode");`
}
```

11.1.5: Is Ethernet enabled?

`boolean isEthernetOpen()`

Return value:

- boolean: true indicates it's enabled, false indicates it's disabled.

Example code:

```
boolean isOpen = mAPI.isEthernetOpen();
```

11.1.6: Switch Ethernet

`void setEthernetState(boolean open)`

parameter:

- open: true enables Ethernet, false disables Ethernet.

Example code:

```
mAPI.setEthernetState(true);
```

12: System Time

12.1: Setting the System Time

`void setSystemTime(long timeInMills)`

parameter:

- timeInMills: Indicates the system time to be set, in milliseconds.

Example code:


```
// Set the system time to 2021-09-28 06:0:0 //
```

Note that the Calendar class starts the month from

0 and the day from 1. Calendar calendar = Calendar.getInstance(Locale.getDefault());

```
calendar.set(2021, 10, 28, 6, 0, 0);
```

```
mAPI.setSystemTime(calendar.getTimeInMillis());
```

12.2: Set a timer for power on/off

```
void setPowerOnOffAlarmCycle(int type,int[] timeOff, int[] timeOn)`
```

parameter:

- type: The power on/off type needs to be set.
- timeOff: The timeout period for the system to be shut down.
- timeOn: The scheduled power-on time that needs to be set.

Example code:

```
// type = 1 Sets the power on/off time. This only takes effect once ; the user must manually reset it via the app each time the device is
```

```
powered on. // type = 3 Sets the power on/off time on a weekly basis. The system saves and applies the settings after completion. Resetting to factory settings or flashing firmware requires resetting.
```

```
int type = 1; int[]
```

```
weekdays = null; // This value can be null or set arbitrarily. When type is not 3, weekdays is invalid. int[] poweroff =
```

```
{2017,3,16,20,30}; // That is , power off at 20:30 on 2017/3/16, hour value 0-23, minute value 0-59 int[] poweron = {2017,3,17,8,35}; //
```

```
That is , power on at 8:35 on 2017/3/17, hour value 0-23, minute value
```

```
mAPI.setPowerOnOffAlarmCycle(type,weekdays,poweroff,poweron);
```

```
int type = 3; int[]
```

```
weekdays={1,3,4,5};//Sets the scheduled power on/off for Monday, Wednesday, Thursday, and
```

```
Friday. Required. int[] poweroff={20,30};//Sets the power off time to 20:30, with hour values from
```

```
0-23 and minute values from 0-59. int[] poweron={8,35};//Sets the power on time to 8:35, with
```

```
hour values from 0-23 and minute values from 0-59. mAPI.setPowerOnOffAlarmCycle(type,weekdays,poweroff,poweron);
```

12.3: Cancel timed power on/off

```
void cancelPowerOnOffAlarm()
```

Example code:

```
mAPI.cancelPowerOnOffAlarm();
```

12.4: Network Synchronization Time

12.4.1: Whether to use network time synchronization

```
void isEnableNetworkProvidedTime()
```

Example code:

```
boolean isEnable = mAPI.isEnableNetworkProvidedTime();
```

12.4.2: Setting Network Synchronization Time

```
void setEnableNetworkProvidedTime(boolean enable)
```

Example code:

```
// Sets whether to allow network time synchronization; true for allow, false for disallow.  
mAPI.setEnableNetworkProvidedTime(true);
```

13: Hardware Module

13.1: GPIO

The API in this section applies to all devices controlled via GPIO. Examples include IO1, relays, LEDs, and USB power supplies (GPIO ports control USB power supply).

Note the peripherals supported by the board, such as relays, LEDs, GPIO ports, etc. You need to consult the specific board specifications. The parameters to be passed during use can be found in our company's specifications.

For detailed implementation information regarding the internal member variables within the provided YNHAPI.jar, please refer to the YNHCommonApiAPK source code provided by our company.

The following are the operable peripheral I/O ports (some boards do not support them, and calling them may not have an effect). See YNHAPI.jar for details.

```
const int GPIO_1           = 0;  
const int GPIO_2           = 1;  
const int GPIO_3           = 2;  
const int GPIO_4           = 3;  
const int GPIO_5           = 4;  
const int GPIO_6           = 5;  
const int GPIO_7           = 6;  
const int GPIO_8           = 7;  
const int GPIO_9           = 8;  
const int GPIO_10          = 9;  
const int GPIO_11          = 10;  
const int GPIO_12          = 11;  
const int GPIO_13          = 12;  
const int GPIO_14          = 13;  
const int GPIO_15          = 14;  
const int LIGHT_RED        = 15; Red fill light  
const int LIGHT_GREEN      = 16; Green fill light  
const int LIGHT_BLUE       = 17; Blue fill light
```

```
const int LIGHT_WHITE = 18; White fill light
const int LIGHT_INFRARED = 19; USB camera infrared fill light
const int RELAY = 20; Relay
const int BELL = 21; doorbell
const int CASHBOX_0 = 22; First money box
const int CASHBOX_1 = 23; The second money box
const int LIGHT_BOARD_BLUE = 24; Onboard blue LED
const int USB_1 = 25; First USB powered
const int USB_2 = 26; Second USB power supply
const int USB_3 = 27; Third USB power supply
const int LIGHT_BOARD_RED = 28; Onboard red light
const int LIGHT_CAMERA_EXTERNAL_INFRARED = 29; MIPI camera infrared fill light
const int LIGHT_BOARD_YELLOW = 30;
```

13.1.1: Obtaining GPIO Status

```
GpioState getGpioState(int index)
```

parameter:

- `index`: Indicates the GPIO whose status needs to be retrieved. For example, `GPIO_1` represents IO1, `RELAY` represents a relay, `LIGHT_RED` represents a red light, etc. (Details...)

You can refer to the YNHAPI.jar provided by our company.

Example code:

```
if (mAPI.getGpioState(YNHAPI.GPIO_1) == YNHAPI.GpioState.HIGH) {
    Log.d(TAG, "GPIO1 is high");
}
```

13.1.2: Setting GPIO Status

```
boolean setGpioState(int index, GpioState state)
```

parameter:

- `index`: Indicates the GPIO whose status needs to be set. For example, `GPIO_1` represents IO1, `RELAY` represents a relay, `LIGHT_RED` represents a red light, etc. (Details...)

You can refer to the YNHAPI.jar provided by our company.

- `state`: The state to be set. For example, `GpioState.HIGH` means setting it to high level.

Return value:

- `boolean`: true indicates success, false indicates failure.

Example code:

```
if (mAPI.setGpioState(YNHAPI.GPIO_1, YNHAPI.GpioState.HIGH)) {
    Log.d(TAG, "Successfully set high level");
}
```

13.1.3: Obtaining GPIO Mode

GpioMode getGpioMode(int index)

parameter:

- index: Indicates the GPIO whose mode needs to be retrieved. For example, GPIO_1 represents IO1, RELAY represents a relay, and LIGHT_RED represents a red light. For details, please refer to YNHAPI.jar provided by our company.

Return value:

- GpioMode: Indicates the GPIO mode being acquired. For example, GpioMode.INPUT indicates the input mode.

Example code:

```
YNHAPI.GpioMode gpioMode = mAPI.getGpioMode(YNHAPI.GPIO_1); if
(gpioMode == GpioMode.INPUT)
{ Log.d(TAG, "GPIO1 is in input mode");
}
```

13.1.4: Setting GPIO Mode

boolean setGpioMode(int index, GpioMode mode)

parameter:

- index: Indicates the GPIO index for which the mode needs to be set. For example, GPIO_1 represents IO1, RELAY represents a relay, and LIGHT_RED represents a red light. For details, please refer to YNHAPI.jar provided by our company.
- mode: Indicates the mode to be set. For example, GpioMode.INPUT sets it to input mode.

Return value:

- boolean: true indicates success, false indicates failure.

Example code:

```
if (mAPI.setGpioMode(YNHAPI.GPIO_1, YNHAPI.GpioMode.INPUT)) {
    Log.d(TAG, "Successfully set input mode");
}
```

13.1.4: Listening for GPIO state changes

synchronized void listenGpio(int index, GpioListenerCallback callback)

This function checks the GPIO state every second after being called. If there is a change, it notifies the application. Therefore, this function is suitable for scenarios where the GPIO state can be maintained for a period of time after a change. It is not suitable for detecting short button presses or changes within one second. For such scenarios, getGpioState can be used directly, and the period can be controlled by the customer.

parameter:

- `index`: Indicates the GPIO index whose state change needs to be monitored. For example, `GPIO\_1` represents IO1, `RELAY` represents a relay, and `LIGHT\_RED` represents a red light.

For details regarding the light source, please refer to the YNHAPI.jar provided by our

- **company.** `callback`: The callback function to be listened to.

Example code:

```
YNHAPI.GpioListenerCallback gpioListenerCallback = new YNHAPI.GpioListenerCallback() {  
    public void onChanged(int index, int oldValue, int newValue) {  
        // This function is called when the state changes. //  
  
        The parameter index is the GPIO index value, indicating which GPIO level state has  
        changed. // The parameter oldValue represents the old  
        level state. // The parameter newValue represents the new level state.  
    }  
};  
mAPI.listenGpio(YNHAPI.GPIO_1, gpioListenerCallback);
```

13.1.5: Cancel listening for GPIO state changes

```
synchronized void unlistenGpio(int index, GpioListenerCallback callback)
```

parameter:

- **`index`:** Indicates the GPIO index whose status change needs to be canceled. For example, `GPIO\_1` represents IO1, `RELAY` represents a relay, and `LIGHT\_RED` represents a red light. For details, please refer to the YNHAPI.jar provided by our company.
- **callback:** The callback that has been set for the listener.

Example code:

```
// Listen for GPIO1 state  
changes mAPI.listenGpio(YNHAPI.GPIO_1, gpioListenerCallback);  
  
// When it is no longer necessary to listen for changes in the GPIO1  
level, call unlistenGpio() mAPI.unlistenGpio(YNHAPI.GPIO_1, gpioListenerCallback);
```

13.2: Setting LED light brightness

```
void setLightBrightness(int index, int brightness)
```

parameter:

- **index:** The GPIO index corresponding to the LED. For example, **LIGHT_RED**, **LIGHT_GREEN**, **LIGHT_WHITE**. For details, please refer to YNHAPI.jar provided by our company.
- **brightness:** Brightness value, ranging from [0, 255].

Example code:

```
// Set the brightness of the  
red light to 80% mAPI.setLightBrightness(YNHAPI.LIGHT_RED, 204);
```

13.3: Wiegand

13.3.1: Obtaining the Wiegand Pattern

WiegandMode readWiegandMode()

Return value:

- WiegandMode: The Wiegand mode to retrieve. For example, WiegandMode.INPUT represents the input mode.

Example code:

```
YNHAPI.WiegandMode wiegandMode = mAPI.readWiegandMode(); if
(wiegandMode == YNHAPI.WiegandMode.INPUT)
    { Log.d(TAG, "Wiegand mode is input mode");
    }
```

13.3.2: Setting Wiegand Mode

void writeWiegandMode(WiegandMode mode)

parameter:

- mode: The Wiegand mode to be set. For example, WiegandMode.INPUT means setting it to input mode.

Example code:

```
mAPI.writeWiegandMode(YNHAPI.WiegandMode.INPUT);
```

13.3.3: Reading Wiegand

Wiegand reads are divided into synchronous blocking reads and asynchronous callback reads.

13.3.4: Synchronous Blocking Read

long readWiegand()

Return value:

- long : The Wiegand number to be read.

Example code:

```
CompletableFuture.supplyAsync(() -> mAPI.readWiegand()) // Asynchronous thread
    read.whenComplete ((code, throwable) -> { //
        Main thread
        updates runOnUiThread(() -> mTextView.setText(String.valueOf(code)));
    });
```

13.3.5: Asynchronous Read

```
void readWiegandAsync(WiegandCallback callback)
```

parameter:

- callback: Callback for asynchronous reading of Wiegand.

Example code:

Note that you must call it in the following way. If you need to read the Wiegand card number asynchronously multiple times, you must call readWiegandAsync in the onSuccess or onFailure callback function.

```
private YNHAPI.WiegandCallback mCallback = new YNHAPI.WiegandCallback() {
    @Override
    public void onSuccess(long code) {
        // Successfully read the Wiegand number; the parameter code represents the Wiegand number.

        // If you need to read Wiegand repeatedly, you need to call
        readWiegandAsync mAPI.readWiegandAsync(mCallback);
    }

    @Override
    public void onFailure() { // Reading
        failed

        // If you need to read Wiegand repeatedly, you need to call
        readWiegandAsync mAPI.readWiegandAsync(mCallback);
    }
};
mAPI.readWiegandAsync(mCallback);
```

Note that the callback occurs on the worker thread.

13.3.6: Writing about Wiegand

```
void writeWiegand(WiegandFormat format, long code)
```

parameter:

- format: Wiegand format. For example, WiegandFormat.FORMAT_26 indicates 26-bit Wiegand format.
- code: Wiegand code.

Example code:

```
mAPI.writeWiegand(YNHAPI.WiegandFormat.FORMAT_26, 12345678);
```

14: Functional Reference Code

14.1: Cash Box Function

The cash drawer function involves operating GPIO. First, the GPIO state is pulled low, and after 500ms, it is pulled high again.

Example code:

```
CompletableFuture.runAsync() ->
    { mYNHAPI.setGpioState(YNHAPI.CASHBOX_0, YNHAPI.GpioState.LOW); try {

        TimeUnit.MILLISECONDS.sleep(500);
    } catch (InterruptedException e)
        { e.printStackTrace();

    } }).thenAccept(u -> {
    mYNHAPI.setGpioState(YNHAPI.CASHBOX_0, YNHAPI.GpioState.HIGH);
});
```

14.2: Executing commands with root privileges

Example code:

```
boolean executeCmd(String content) throws IOException, InterruptedException { boolean ret = false;
    Process p =
    Runtime.getRuntime().exec("su"); if (p != null) { PrintWriter
    pw = new
        PrintWriter(p.getOutputStream(), true); pw.println(content); pw.println("exit"); ret =
        p.waitFor() == 0;

    }
    return ret;
}
```